

# Experimental Evaluation and Optimization of Real-Time Scheduling on STM32

Vladimir Toporkov, Artem Burmyakov  
Innopolis University

## I. INTRODUCTION

### A. Contribution

Our contributions are about evaluation of schedulers for STM32 microcontroller:

- we first implement Super Loop scheduler,
- then we implement chunked Earliest Deadline First (EDF) scheduler, and finally
- we evaluate their performance on a real STM32 hardware.

## II. BACKGROUND ON STM32

The experimental platform consists of a uniprocessor STM32F767 microcontroller and external sensors connected through its hardware peripherals, as shown in Fig. 1.

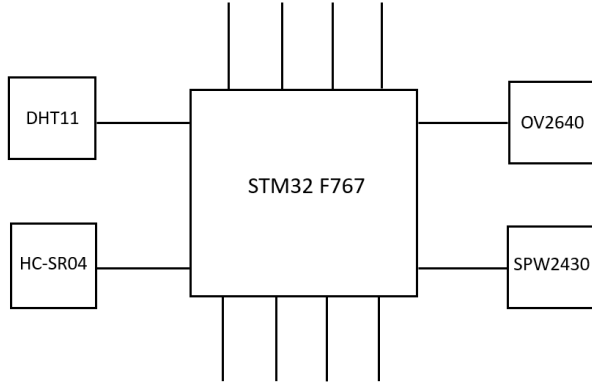


Fig. 1. STM32 hardware platform and connected sensors.

The hardware components are presented in Table I.

TABLE I  
EXPERIMENTAL HARDWARE PLATFORM.

| Component | Purpose                              |
|-----------|--------------------------------------|
| STM32F767 | Scheduling and task execution        |
| DHT11     | Temperature and humidity measurement |
| HC-SR04   | Distance measurement                 |
| OV2640    | Image capturing                      |
| SPW2430   | Acoustic signal acquisition          |

## III. SYSTEM MODEL

We model the workload by a set of periodic tasks  $\mathcal{T} = \{\tau_1, \dots, \tau_n\}$  executed on a uniprocessor. We consider only implicit deadlines, i.e.,  $D_i = T_i$ . Each task  $\tau_i = (C_i, T_i = D_i)$  generates a sequence of jobs  $J_i^j$ , where  $j$  denotes the scheduled

release index. Consecutive scheduled releases satisfy  $R_i^{j+1} = R_i^j + T_i$ .

TABLE II  
SYSTEM MODEL NOTATION.

| Symbol                     | Meaning                                                       |
|----------------------------|---------------------------------------------------------------|
| $\mathcal{T}, n$           | Task set and number of tasks                                  |
| $\tau_i$                   | Task $i$                                                      |
| $C_i$                      | Execution time of a job of $\tau_i$                           |
| $T_i$                      | Task period                                                   |
| $D_i = T_i$                | Relative deadline, equal to the period                        |
| $J_i^j$                    | Job associated with the $j$ -th scheduled release of $\tau_i$ |
| $R_i^j$                    | Scheduled release time                                        |
| $f_i^j$                    | Completion time of an executed job                            |
| $d_i^j = R_i^j + D_i$      | Absolute deadline                                             |
| $U = \sum_{i=1}^n C_i/T_i$ | Task-set utilization                                          |

Table II summarizes the notation. Release indices include skipped activations, for which no completion time is defined.

- A schedule specifies which job executes on the processor at each time instant, or whether the processor executes no job.
- Preemption interrupts a job before completion to execute another job. The interrupted job may resume later.
- A completed job  $J_i^j$  has missed its deadline if  $f_i^j > d_i^j$ .

## IV. SCHEDULING PRINCIPLES

### A. Super Loop Scheduler

The considered Super Loop scheduler repeatedly scans tasks in a fixed order and executes each selected job non-preemptively. To illustrate its behavior, let us consider the two periodic tasks in Table III. Both tasks first release at  $t = 0$ .

TABLE III  
TASK SET USED IN THE SUPER LOOP EXAMPLE.

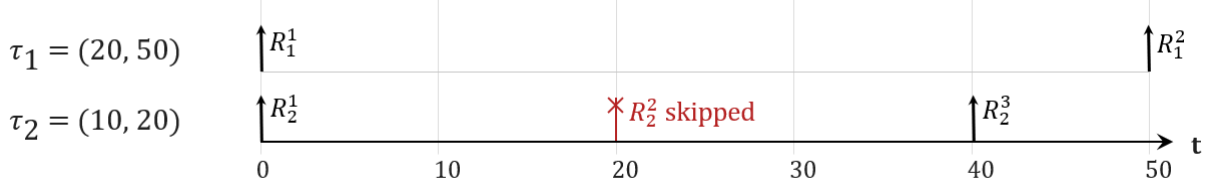
| Task     | $C_i$ | $T_i$ | $D_i$ |
|----------|-------|-------|-------|
| $\tau_1$ | 20    | 50    | 50    |
| $\tau_2$ | 10    | 20    | 20    |

The resulting Super Loop schedule is shown in Fig. 2.

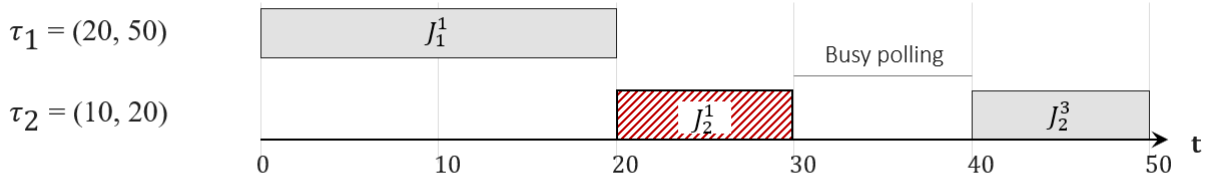
Let  $t^{cur}$  denote the current system time and  $t_i^{next}$  the next scheduled release time of task  $\tau_i$ . During execution of  $J_i^j$ , the release variable  $t_i^{next}$  retains  $R_i^j$ . At completion,  $f_i^j$  denotes the completion time, and a deadline miss is recorded if  $f_i^j > d_i^j$ . The scheduler then advances  $t_i^{next}$  by  $T_i$ . While  $t_i^{next} <$

$$\tau_i = (C_i, D_i = T_i)$$

### Scheduled Releases



### Super Loop execution scenario



Absolute Deadline: 20 Completion: 30

$J_j^i$  – job execution     $R_j^i$  – scheduled release time

Fig. 2. Super Loop execution scenario for the task set in Table III.

$f_i^j$ , the scheduled release at  $t_i^{next}$  is counted as skipped and  $t_i^{next}$  is advanced by  $T_i$  again. A release at  $t_i^{next} = f_i^j$  is retained.

Algorithm 1 summarizes the illustrated scheduling logic. Since  $R_i^1 = 0$ , the release index is  $j = 1 + t_i^{next}/T_i$ , including skipped releases. The clock used by NOW starts at zero.

The corresponding scheduling state is shown in Table IV.

TABLE IV  
SUPER LOOP SCHEDULING STATE FOR FIG. 2.

| $t^{cur}$ | $t_1^{next}$ | $t_2^{next}$ | Deadline miss check | Release skip check |
|-----------|--------------|--------------|---------------------|--------------------|
| 0         | 0            | 0            | –                   | –                  |
| 20        | 50           | 0            | $20 < R_1^1 + 50$   | $20 < R_1^2$       |
| 30        | 50           | 40           | $30 > R_2^1 + 20$   | $30 > R_2^2$       |
| 40        | 50           | 40           | –                   | –                  |
| 50        | 50           | 60           | $50 < R_2^3 + 20$   | $50 < R_2^4$       |

In Fig. 2,  $J_1^1$  executes from  $t = 0$  to  $t = 20$ . Then,  $J_2^1$  executes until  $t = 30$ , missing its deadline at  $t = 20$ . At completion, the release scheduled for  $R_2^2 = 20$  is counted as skipped and  $t_2^{next}$  advances to  $R_2^3 = 40$ . Thus, no  $J_2^2$  is executed.

#### B. Earliest Deadline First

Unlike Super Loop, Earliest Deadline First (EDF) selects jobs according to their absolute deadlines.

We first consider preemptive EDF with immediate preemption. At each scheduling decision, the ready job with the

#### Algorithm 1 Super Loop Scheduler

```

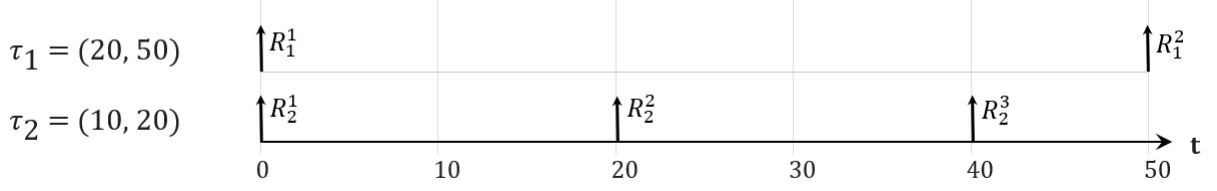
1: function SUPERLOOP( $\mathcal{T}$ )
2:   for all  $\tau_i \in \mathcal{T}$  do
3:      $t_i^{next} \leftarrow 0$ 
4:   end for
5:   while true do
6:     for  $i \leftarrow 1$  to  $n$  do ▷ Fixed task order
7:        $t^{cur} \leftarrow \text{NOW}$ 
8:       if  $t^{cur} \geq t_i^{next}$  then
9:          $j \leftarrow 1 + t_i^{next}/T_i$ 
10:         $R_i^j \leftarrow t_i^{next}$ 
11:        EXECUTE( $J_i^j$ ) ▷ Non-preemptive
12:         $f_i^j \leftarrow \text{NOW}$ 
13:        if  $f_i^j > R_i^j + D_i$  then
14:          RECORDMISS( $J_i^j$ )
15:        end if
16:         $t_i^{next} \leftarrow t_i^{next} + T_i$ 
17:        while  $t_i^{next} < f_i^j$  do
18:          RECORDSKIP( $\tau_i, t_i^{next}$ )
19:           $t_i^{next} \leftarrow t_i^{next} + T_i$ 
20:        end while
21:      end if
22:    end for
23:  end while
24: end function

```

earliest absolute deadline  $d_i^j$  is selected (Section III). If a newly released job has an earlier absolute deadline than the running

$$\tau_i = (C_i, D_i = T_i)$$

### Scheduled Releases



### EDF execution scenario

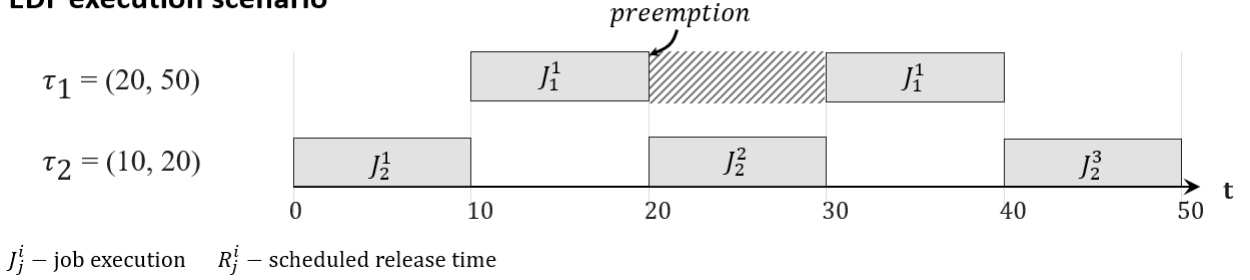


Fig. 3. EDF execution scenario for the task set in Table III. Hatching indicates a preempted job awaiting resumption.

job, it preempts the running job.

Fig. 3 illustrates EDF scheduling for the task set in Table III.

At  $t = 20$ ,  $J_2^2$  is released with  $d_2^2 = 40 < d_1^1 = 50$  and preempts  $J_1^1$ . After  $J_2^2$  completes at  $t = 30$ ,  $J_1^1$  resumes and completes at  $t = 40$ .

In the STM32 implementation, synthetic jobs execute in chunks of maximum duration  $\Delta$ , with job selection repeated after each chunk. An earlier-deadline job released during a chunk may therefore wait until the chunk completes. Sensor jobs use separate staged handlers.

For illustration, Fig. 4 uses  $\Delta = 4$  ms. Job  $J_2^2$  is released at  $t = 20$  ms and preempts  $J_1^1$  at the next chunk boundary  $t = 22$  ms. Thus, the preemption delay is  $\delta = 2$  ms. The final chunk of  $J_2^2$  lasts 2 ms, since the preceding two chunks account for 8 ms of its execution time  $C_2 = 10$  ms.

Ignoring scheduling overhead, the delay until the next chunk boundary satisfies  $0 \leq \delta \leq \Delta$ .

Algorithm 2 summarizes chunked EDF for synthetic jobs, assuming first releases at  $t = 0$  as in the example. The set  $\mathcal{R}$  contains jobs admitted for execution but not yet completed. The flag  $active_i$  is true while a job of  $\tau_i$  belongs to  $\mathcal{R}$ , including before its first execution. Equal deadlines are resolved by task order. EXECUTECHUNK executes at most  $\Delta$  of the selected job's remaining execution and preserves its progress.

### Chunked EDF

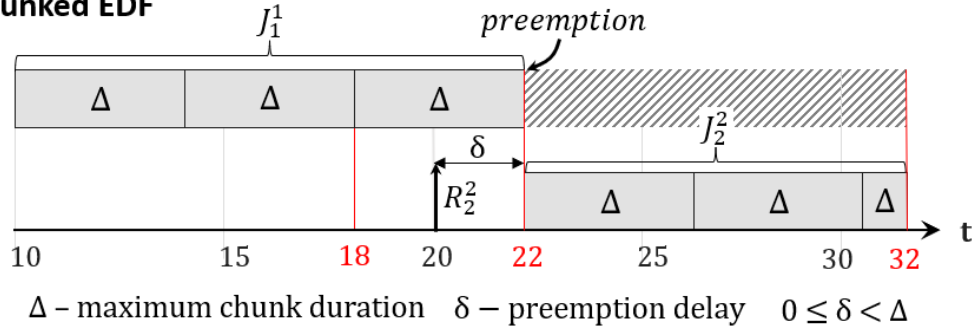


Fig. 4. Chunk-bounded preemption in the EDF implementation.

---

**Algorithm 2** Chunked EDF Scheduler

---

```
1: function CHUNKEDEDF( $\mathcal{T}, \Delta$ )
2:    $\mathcal{R} \leftarrow \emptyset$ 
3:   for all  $\tau_i \in \mathcal{T}$  do
4:      $t_i^{next} \leftarrow 0$ 
5:      $active_i \leftarrow false$ 
6:   end for
7:   while true do
8:      $t^{cur} \leftarrow \text{NOW}$ 
9:     for all  $\tau_i \in \mathcal{T}$  do
10:      if  $\neg active_i \wedge t^{cur} \geq t_i^{next}$  then
11:         $j \leftarrow 1 + t_i^{next} / T_i$ 
12:         $R_i^j \leftarrow t_i^{next}$ 
13:         $d_i^j \leftarrow R_i^j + D_i$ 
14:         $\mathcal{R} \leftarrow \mathcal{R} \cup \{J_i^j\}$ 
15:         $active_i \leftarrow true$ 
16:      end if
17:    end for
18:    if  $\mathcal{R} \neq \emptyset$  then
19:       $J_i^j \leftarrow \text{SELECTEDF}(\mathcal{R})$   $\triangleright$  Earliest deadline
20:       $\text{EXECUTECHUNK}(J_i^j, \Delta)$ 
21:      if  $\text{COMPLETED}(J_i^j)$  then
22:         $f_i^j \leftarrow \text{NOW}$ 
23:        if  $f_i^j > d_i^j$  then
24:           $\text{RECORDMISS}(J_i^j)$ 
25:        end if
26:         $t_i^{next} \leftarrow t_i^{next} + T_i$ 
27:        while  $t_i^{next} < f_i^j$  do
28:           $\text{RECORDSKIP}(\tau_i, t_i^{next})$ 
29:           $t_i^{next} \leftarrow t_i^{next} + T_i$ 
30:        end while
31:         $\mathcal{R} \leftarrow \mathcal{R} \setminus \{J_i^j\}$ 
32:         $active_i \leftarrow false$ 
33:      end if
34:    end if
35:  end while
36: end function
```

---

## V. PERFORMANCE EVALUATION

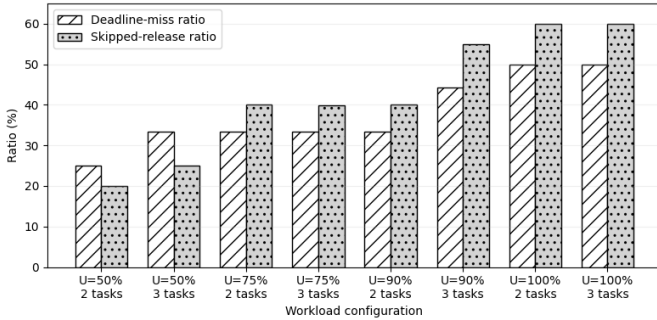


Fig. 5. Per-task deadline-miss and skipped-release ratios under Super Loop. Each configuration specifies the task-set utilization  $U$  and the number of tasks.